

Phase 2 Design Patterns Used

This document lists the design patterns already implemented in the project, why each was used, and which problem each pattern solves.

1) MVC (Model-View-Controller)

- Where used:
 - Models: `app/Models/*`
 - Controllers: `app/Http/Controllers/*`
 - Views: `resources/views/*`
- Why used:
 - Laravel is MVC-native, and this structure keeps request handling, domain data, and UI rendering separated.
- Problem solved:
 - Prevents business/data/UI concerns from being mixed in one place.
 - Improves maintainability by making each layer easier to reason about.

2) Service Layer (Application Services)

- Where used:
 - `app/Services/SubscriptionService.php`
 - `app/Services/BoxProvisioningService.php`
 - `app/Services/DeliveryProvisioningService.php`
 - `app/Services/ReportsService.php`
 - `app/Services/StockAllocationService.php`
 - `app/Services/NotificationDispatchService.php`
 - and other classes under `app/Services/*`
- Why used:
 - Complex business rules are extracted from controllers into dedicated services.

- Problem solved:
 - Eliminates fat controllers and duplicated flow logic.
 - Makes business logic reusable across HTTP controllers, commands, and jobs.

3) Strategy Pattern (Notification Channels)

- Where used:
 - Strategy contract: `app/Services/NotificationChannels/NotificationChannel.php`
 - Concrete strategies:
 - `app/Services/NotificationChannels/DatabaseNotificationChannel.php`
 - `app/Services/NotificationChannels/LogNotificationChannel.php`
 - Selector/dispatcher: `app/Services/NotificationChannels/ChannelDispatcher.php`
- Why used:
 - Notification delivery behavior changes by channel type.
- Problem solved:
 - Avoids large conditional blocks for each channel in one class.
 - Enables extending with new channels without modifying existing channel implementations.

4) State Machine Pattern (Delivery Lifecycle)

- Where used:
 - `app/Services/DeliveryStateTransitionService.php`
- Why used:
 - Delivery statuses have strict allowed transitions and role-based guard rules.
- Problem solved:
 - Prevents invalid status jumps (for example skipping required steps).
 - Centralizes transition rules so all update paths enforce identical behavior.

5) Policy-Based Authorization

- Where used:
 - Policies:
 - `app/Policies/SubscriptionPolicy.php`
 - `app/Policies/DeliveryPolicy.php`
 - `app/Policies/ClaimPolicy.php`
 - `app/Policies/UserPolicy.php`
 - Registration:
 - `app/Providers/AppServiceProvider.php`
- Why used:
 - Access control is defined per domain model action instead of scattered inline checks.
- Problem solved:
 - Reduces authorization drift and duplicated permission logic.
 - Improves security consistency and testability.

6) Request Validation Object Pattern (FormRequest)

- Where used:
 - `app/Http/Requests/*` (for example `UpdateDriverDeliveryStatusRequest.php` , `ResolveClaimRequest.php` , `ReportFilterRequest.php`)
- Why used:
 - Validation and request authorization are encapsulated in dedicated request classes.
- Problem solved:
 - Removes inline validation clutter from controllers.
 - Standardizes input contracts and error handling across endpoints.

7) Command Pattern (Scheduled/Manual Tasks)

- Where used:

- `app/Console/Commands/RenewSubscriptions.php`
- `app/Console/Commands/ProcessQueuedNotifications.php`
- `app/Console/Commands/SyncTimeBasedStates.php`
- `app/Console/Commands/CheckMissingDeliveries.php`
- Why used:
 - Operational workflows are represented as executable command objects.
- Problem solved:
 - Separates background/ops workflows from web request lifecycle.
 - Makes repeated operations deterministic and automatable.

8) Observer-Like View Composer (Cross-Cutting Header Notifications)

- Where used:
 - `app/Providers/AppServiceProvider.php` (`View::composer('layouts.app', ...)`)
- Why used:
 - Shared header notification state is attached automatically when layout renders.
- Problem solved:
 - Prevents repetitive notification-count query logic in each controller.
 - Ensures one canonical source for header notification binding.